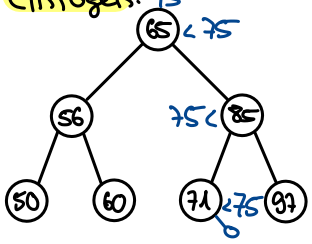
Binäre (Such-)Bäume:

Zweifelsfall immer sym. Vorgänger

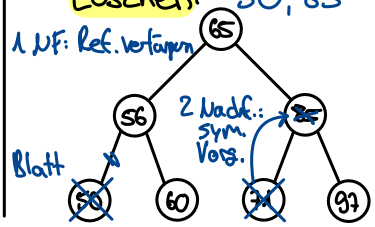
symmetrischer Vorgänger: $e_{max(T_l)}$
größtes Element im linken Teilbaum

symmetrischer Nachfolger: $e_{min(T_r)}$
kleinste Element im rechten Teilbaum

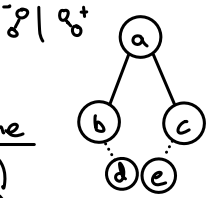
Einfügen: 75



Löschen: 50, 85



AVL-Bäume $h < 1.45 \log_2(n+2) - 1.33$



Einfügen: wie Suchbaum → bf checken

a	b	c	Maßnahme
--	-		R(b)
- -	+		LR(d)
+ +		+	L(c)
+ +		-	RL(e)

Löschen: wie Suchbaum → bf checken → ggf. Maßnahme

Treap: (Key, Priority)

Suchbaum Min-Heap



(Key) (Rotationsheap)

Einfügen: wie Suchbaum → Rotieren bis in Heap (Prio)

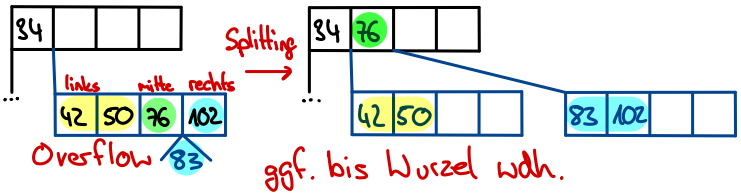
Löschen: Rotation um Nachfolger mit kleinerer Prio.
↓ ↑
ist Blatt? → Blatt aushängen

B-Bäume: Ordnung d (= max. Nachfolger)

	Nachfolger	Elemente
Wurzel	2 bis d	1 bis d-1
Blätter	0	$\lceil \frac{d-1}{2} \rceil$ bis d-1
sonstige Knoten	$\lceil \frac{d}{2} \rceil$ bis d	$\lceil \frac{d-1}{2} \rceil$ bis d-1

alle auf gleicher Ebene

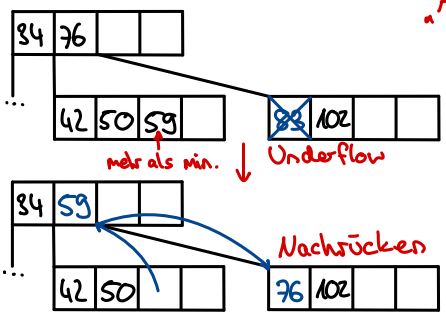
Einfügen: 83



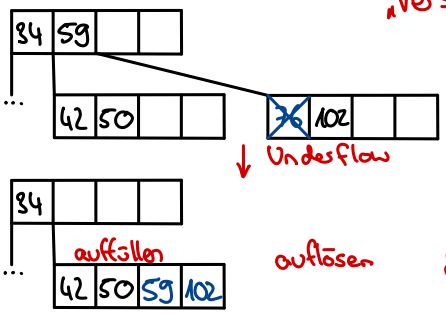
ggf. bis Wurzel wdh.

Löschen I: El. kein Blatt → tausche sym. Nachfolger

Löschen II: 83 Nachbar mehr als Mindestbelegung „Ausgleichen"



Löschen III: 76 Nachbar mit Mindestbelegung „Verschmelzen"



auflösen

ggf. bis Wurzel wdh.

Division: $h(s) = s \bmod m$

✗ m keine 2er-/10er-Potenz, $m \neq 2^k - 1$

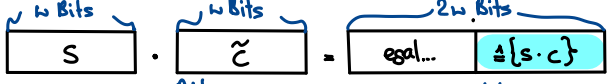
✓ m ist Primzahl, m nicht nah an 2er-Potenz

Multiplikation: $h(s) = \lfloor m \cdot \{s \cdot c\} \rfloor$ $c \in \mathbb{R}, 0 < c < 1$
($\{\}$ gebrochen, $\lfloor\rfloor$ ganzzahlig)

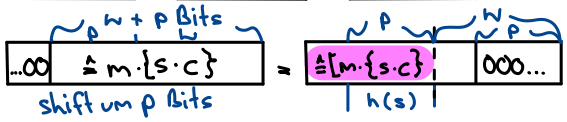
Ganzzahlarithmetik: $s \in [0, 2^w - 1], h(s) \in [0, 2^p - 1], m = 2^p$

1.: s binär, $\tilde{c}$ = erste w Nachkommastellen von c binär

2.:



3.:



Lineares Sondieren: $i(s)_j = (h(s) + j) \bmod m, \; j = 0, \ldots, m-1$

Quadratisches Sondieren:



$$i(s)_j = \left(h(s) + (-1)^{(j+1) \bmod 2} \left\lceil \frac{j}{2} \right\rceil^2 \right) \bmod m, \; j = 0, \ldots, m-1$$

Primzahl $p = 4k + 3, k \in \mathbb{N}_0$ (erreichbare Tabellenplätze)

Doppelhashing: $i(s)_j = (h(s) + j \cdot h^*(s)) \bmod m, \; j = 0, \ldots, m-1$

$h^*(s)$ und m teilerfremd, am besten m Primzahlen

Differenzengleichung 1. Ordnung:

Anfang $X_1 = b$ Rekursion $X_n = a_n X_{n-1} + b_n$, $n \geq 2$

1) $\pi_i = \prod_{j=2}^{i} a_j$ ($n \to j$), $2 \leq i \leq n$, $\pi_1 = 1$

2) $x_n = \pi_n \left(b + \sum_{i=2}^{n} \frac{b_i}{\pi_i} \right)$ ($i \to n$), $n \geq 1$

Partialbruchzerlegung:

$$\frac{4i-2}{i(i+1)} = \frac{A}{i} + \frac{B}{i+1} \quad | \cdot i(i+1)$$

$$4i - 2 = A(i+1) + Bi$$

$$4i - 2 = (A+B)i + A \rightarrow \begin{matrix} 4 = A + B \\ -2 = A \end{matrix} \Big\} \; B = -6$$

Summen-/Produktformeln:

Worst-case: ausschreiben

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \qquad \sum_{i=1}^{n} k = (n-1+1)k$$

$$\prod_{i=1}^{n} i = i! \qquad \prod_{i=1}^{n} k = k^n$$

$$\sum_{i=1}^{n} (k+l) = \sum_{i=1}^{n} k + \sum_{i=1}^{n} l \qquad \prod_{i=1}^{n} k \cdot l = \prod_{i=1}^{n} k \cdot \prod_{i=1}^{n} l$$

$$\sum_{i=1}^{n} i = \sum_{i=2}^{n+1} (i-1) \overset{+x}{\underset{-x}{}} = \sum_{i=1}^{n-1} i - n = \sum_{i=2}^{n} i + 1$$

$H_n = \sum_{i=1}^{n} \frac{1}{i}$, n-te harmonische Summe

	Best	Avg.	Worst
if then else	1 Min(then,else)	1 c_{then} · Anteil + c_{else} · Anteil	1 Max(then,else)
for	Min	Avg.	Max

m = 11, 4-Bit Schlüssel, 3-Bit Hashwerte, $c = (0.101)_2$

$h^k(s) = 1 + (s \bmod (m-1))$

$h(2)$: $0010 \cdot 1010 = 0001\,0100$ (s, $\tilde{c}$)

$h(2) = (010)_2 = 2$

$h(3)$:

$$\begin{array}{r} 0011 \cdot 1010 \\ \hline 0011 \\ \hline 0001\,1110 \end{array}$$

$h(3) = (111)_2 = 7$

$\{g(n) : \mathbb{N} \to \mathbb{R}_+ : \exists c > 0 \; \exists n_0 \in \mathbb{N} \; \forall n \geq n_0 : \ldots$

$O(f(n)) := \ldots g(n) \leq c \cdot f(n) \}$, $g \leq f$

$\Omega(f(n)) := \ldots c \cdot f(n) \leq g(n) \}$, $g \geq f$

$\Theta(f(n)) := \overset{\exists c_1, c_2}{\ldots} c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n) \}$, $g = f$

$o(f(n)) := \ldots g(n) < c \cdot f(n) \}$, $g < f$

$\omega(f(n)) := \ldots g(n) > c \cdot f(n) \}$, $g > f$

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0 \iff g = o(f) \;/\; g < f$$

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \infty \iff g = \omega(f) \;/\; g > f$$

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} \overset{c \neq 0}{=} c \iff g = \Theta(f) \;/\; g = f$$

$$\lim_{n\to\infty} (A(n) \div B(n)) = \lim_{n\to\infty} A(n) \div \lim_{n\to\infty} B(n)$$

$$\lim_{n\to\infty} \frac{A(n)}{B(n)} = \frac{\lim_{n\to\infty} A(n)}{\lim_{n\to\infty} B(n)} \qquad \lim_{n\to\infty} \frac{A(n)}{B(n)} = \lim_{n\to\infty} \frac{A(n)'}{B(n)'}$$

L'Hospital

$$\log_a(n \cdot m) = \log_a(n) + \log_b(m) \qquad \log_a\left(\frac{n}{m}\right) = \log_a(n) - \log_a(m)$$

$$\log_a(n^m) = m \log_a(n) \qquad \log_a(\sqrt[m]{n}) = \frac{1}{m} \log_a(n)$$

$$\log_a(n) = \log_a(b) \log_b(n) \qquad n = a^{\log_a(n)}$$

$$(\log_a(n))' = \frac{1}{n \ln(a)} \qquad (\log_2(\log_2(n)))' = \frac{1}{\log_2(n) \ln(2)} \cdot \frac{1}{n \ln(2)}$$

$$a^n a^m = a^{n+m} \qquad \frac{a^n}{a^m} = a^{n-m}$$

$$a^n b^n = (ab)^n \qquad \frac{a^n}{b^n} = \left(\frac{a}{b}\right)^n$$

$$(a^n)^m = a^{nm} \qquad (a^n)^{\frac{1}{m}} = a^{\frac{n}{m}}$$

L'Hospital hilft meistens

$$\frac{2^{\log_2(\ldots)}}{2^{\log_2(\ldots)}} = 2^{\log_2(\ldots) - \log_2(\ldots)}$$ ($:= e(n)$)

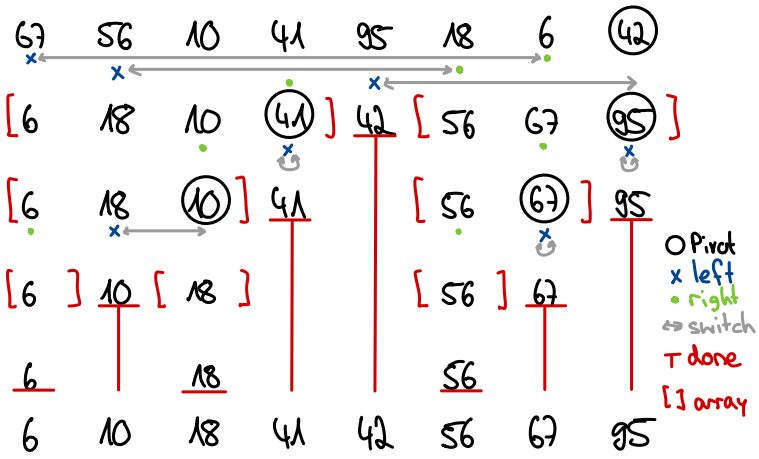
$\lim_{n\to\infty} e(n) = \ldots = z^{\infty/\infty}$

$$\sqrt[n]{a} \cdot \sqrt[n]{b} = \sqrt[n]{ab} \qquad \frac{\sqrt[n]{a}}{\sqrt[n]{b}} = \sqrt[n]{\frac{a}{b}} \qquad \sqrt{n} = n^{\frac{1}{2}},\ \text{Abl.:}\ \frac{1}{2} n^{-\frac{1}{2}}$$

$$\sqrt[m]{\sqrt[n]{a}} = \sqrt[mn]{a} \qquad \sqrt[n]{a^m} = (\sqrt[n]{a})^m \qquad a^n \sqrt{b} = \sqrt{a^n b}$$

$$1 < \log_b(n) < \log_b^m(n) \overset{\sqrt[m]{n}}{<} n < n \log_b(n) < n^k \overset{\text{Polynome einzeln}}{<} 2^{cn} < n!$$

```
void QuickSort(item a, index i, j)
1  item x, index l, r
2  if i < j          // es wird nur sortiert, wenn mindestens 2 Elemente im Array
3  then x ← a[j], l ← i, r ← j − 1   // Pivotauswahl: a[j](=x) ist das Pivotelement
4       while true do
5         while a[l] < x do l ← l + 1
6         while a[r] > x do r ← r − 1   // Bilde zwei Teilarray F1, F2:
7         if l < r                      // für alle f1 in F1 gilt: f1 <= x
8         then exchange a[l] and a[r]   // für alle f2 in F2 gilt: f2 >= x
9              l = l + 1, r = r − 1
10        else break
11 exchange a[l] and a[j]   // Tausche das Pivotelement an seine Rangposition*)
12 [ ] QuickSort(a, i, l − 1)   // Rekursion: Sortiere F1
13 [ ] QuickSort(a, l + 1, j)   // Rekursion: Sortiere F2
```



SelectionSort

1) Minimum suchen
2) mit Anfang tauschen
3) Anfang + 1

InsertionSort

1) 2. Element nach links, bis korrekt in sort. Bereich
2) bis letztes El. wdh.

Binäre Suche

$\lfloor \frac{l+r}{2} \rfloor = e$ { a[e] > x : r = e − 1 ; a[e] < x : l = e + 1 } | bis a[e] = x oder l > r

Datenstrukturen

Struktur	Suchen	Einfügen	Löschen
Unsortiertes Array	O(n)	O(1)	O(n)
Sortiertes Array	O(log n)	O(n)	O(n)
Binärer Suchbaum	O(n)	O(n)	O(n)
AVL-Baum	O(log n)	O(log n)	O(log n)
B-Baum	$O(\log_d n)$	$O(\log_d n)$	$O(\log_d n)$
Hash-Tabelle	O(1) avg	O(1) avg	O(1) avg

Algorithmen

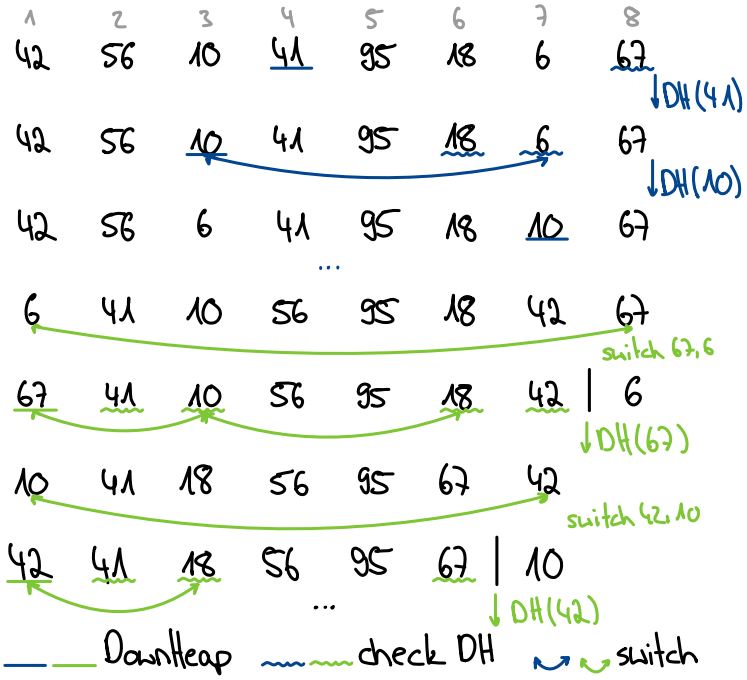
Algorithmus	Best Case	Average Case	Worst Case
Quicksort	O(n log n)	O(n log n)	$O(n^2)$
Mergesort	O(n log n)	O(n log n)	O(n log n)
Heapsort	O(n log n)	O(n log n)	O(n log n)
BFS/DFS	O(|V| + |E|)	O(|V| + |E|)	O(|V| + |E|)
Dijkstra	O(|E| log |V|)	O(|E| log |V|)	O(|E| log |V|)

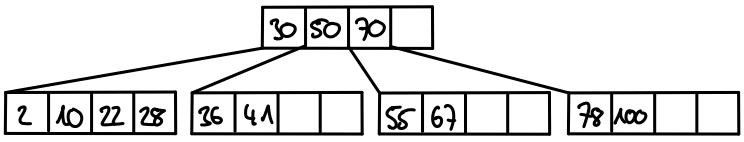
DownHeap (item a, left l, right r)

1) vergleiche a[left] mit a[left*2], a[left*2+1]
2) tausche mit kleinerem
3) Wdh. bis right > left*2 (Blatt)

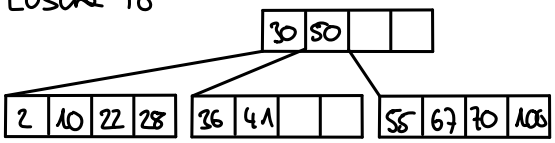
```
void HeapSort(item a)
1  index : l, r
2  for l ← n div 2 downto 1 do
3    DownHeap(a, l, n)          } Build Heap
4  for r ← n downto 2 do
5    exchange a[1] and a[r]     } Sort
6    DownHeap(a, 1, r − 1)
```

Sort: 1) mit Start tauschen 2) Downheap never Start

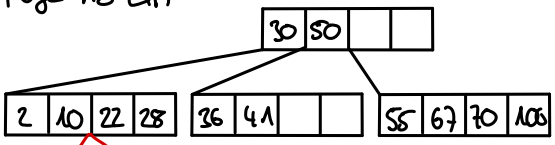


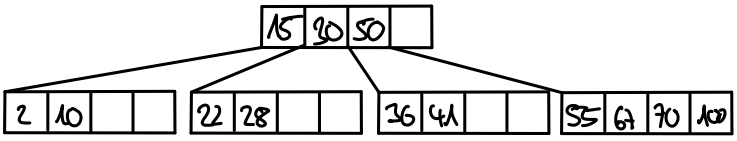


Lösche 78



Füge 15 ein





$\tilde{c} = 1011\ 1011$ $m = 16 - 2^4 = 10000$ $w = 8$
$3 = 0011$

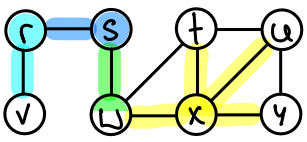
$h(3) = \frac{1011\,1011 \cdot 0011}{}$ $\lfloor s \cdot \tilde{c} \rfloor$ $m \cdot \lfloor s \cdot \tilde{c} \rfloor$

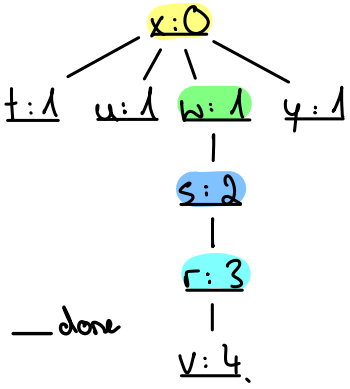
$+\ 1011\,1011\ 0$
$10\,0011\,0001$
$h(3) = 3$

Graph $G = (V, E)$, $E = \{\{u,v\}\}$ (ungerichtet) oder $\{(u,v)\}$ (gerichtet)

Breitensuche: BFS

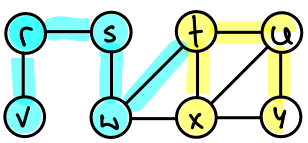
(eher ungerichtet)

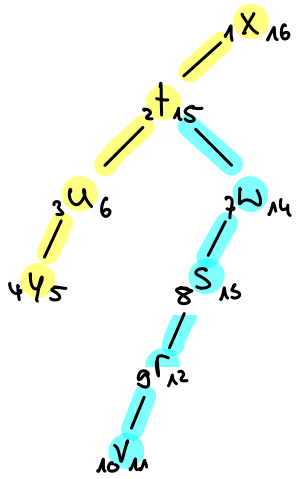




Tiefensuche: DFS

(eher gerichtet)



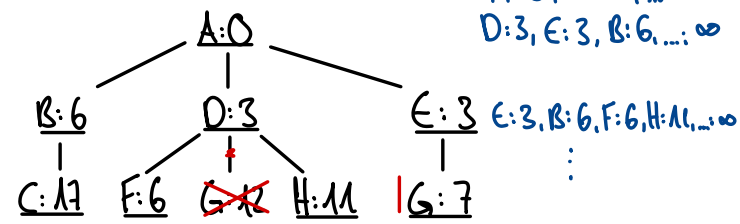


Dijkstra: Single-Source Shortest Path

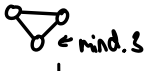
BFS nach Prio. (Priority Queue), Distanz zu Wurzel addieren

A:0, B:∞, ...:∞

D:3, E:3, B:6, ...:∞



Zyklenerkennung





ungerichtete Graphen:

BFS + adj. Knoten { nicht INITIAL und nicht Elternkn.
→ zurück bis LCA

gerichtete Graphen:

bzw. Vorfahre

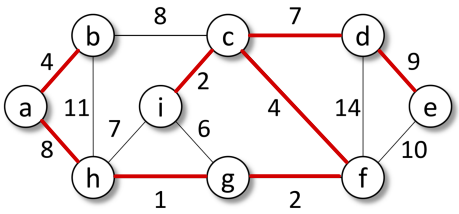
DFS + adj. Knoten PENDING
→ Rückwärtskante

Freier / ungerichteter Baum

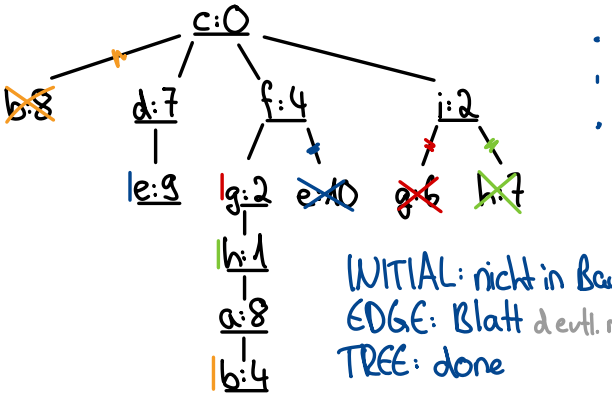
G azyklisch ∧ zusammenhängend

Gerüst / Spannbaum

Teilgraph, freier Baum ∧ alle Knoten



Prim: Dijkstra, aber nur Distanz zu Baum



INITIAL: nicht in Baum
EDGE: Blatt d evtl. nicht optimal
TREE: done

Topologisches Sortieren: Knoten vor Nachfolgern

DFS + alle Nachfolger besucht → bei Listenende einf.

SCC's: Zusammenhangskomponente

Jeder Knoten von jedem erreichbar

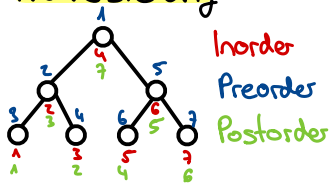
1. DFS(G)
2. compute G^T (transponiert) A: B, B: ... → A: ..., B: A
3. DFS(G^T) nach absteigender Endtime
   ↳ einzelne Bäume ≙ jeweils SCC

Kruskal: günstigste Kante, die neue Knoten verbindet nehmen

[g,h], [c,i], [f,g], [a,b], [c,f], [c,d], [a,h] ← irgendwie so

oder alle Kanten nach Prio aufzählen, alle Knoten drin → Stop

Traversierung



Inorder
Preorder
Postorder

	LTR	Wurzel	RTR
In:	a	b	c, d, e, f, g
Pre:	d	b, a, c	f, e, g
Post:	a, c	b	e, g, f, d

Pre: ... d, b
Post: ..., f, b
↳ 2 Kinder

Heaps

Rotationheap: hochrotieren → bis Heap-Bedingung

Downheap: Knoten mit kleinerem Kind tauschen

Buildheap: DH von 1. Blatt bis Wurzel